RICE UNIVERSITY

T1 · GPU and framework execution

Write one fused Triton kernel, check it against PyTorch, use it in a supplied model, and compare eager and compiled execution.

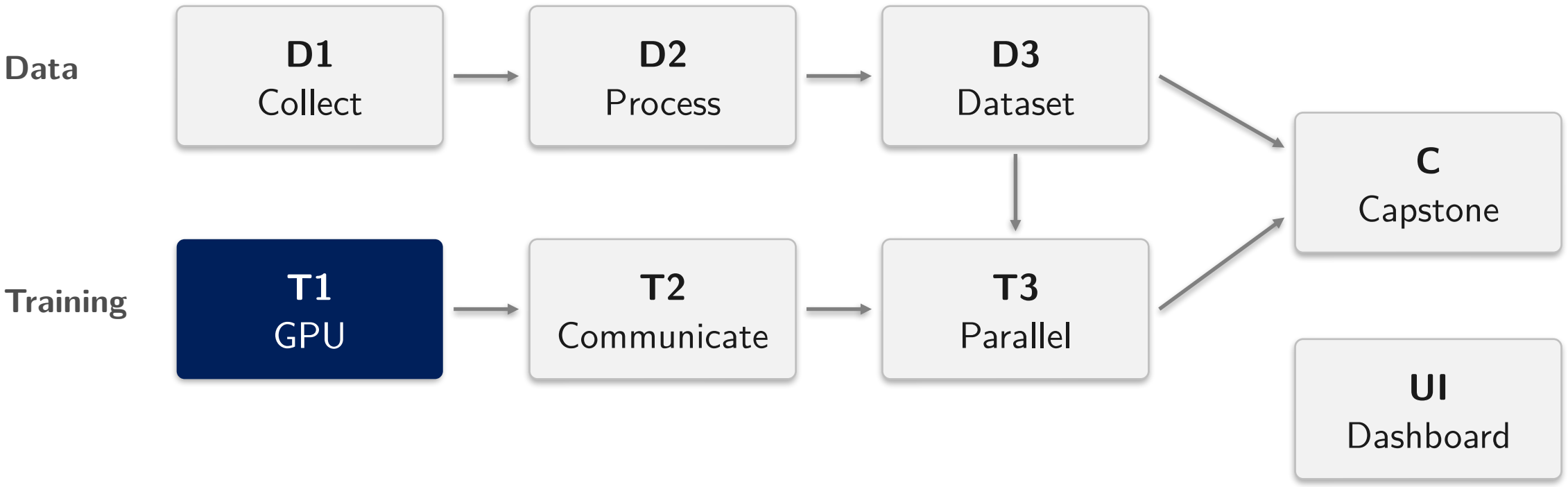
COMP 329 course staff

Rice University

Spring 2027 · Assignment T1

Release week 8 · Checkpoint week 9 · Due week 10 · 7% of grade

Where this assignment fits



In our pipeline

This assignment: `runs/inputs/bpe` → **T1** → `assignment-result.json`

What you will build, and why

Make tanh(x + b) one GPU kernel instead of two, prove it is correct, time it honestly, and explain where the time goes in a real model.

1

Memory is the bottleneck

Simple element-wise operations wait on HBM, not arithmetic. Fusion saves memory traffic.

2

Timing GPUs is tricky

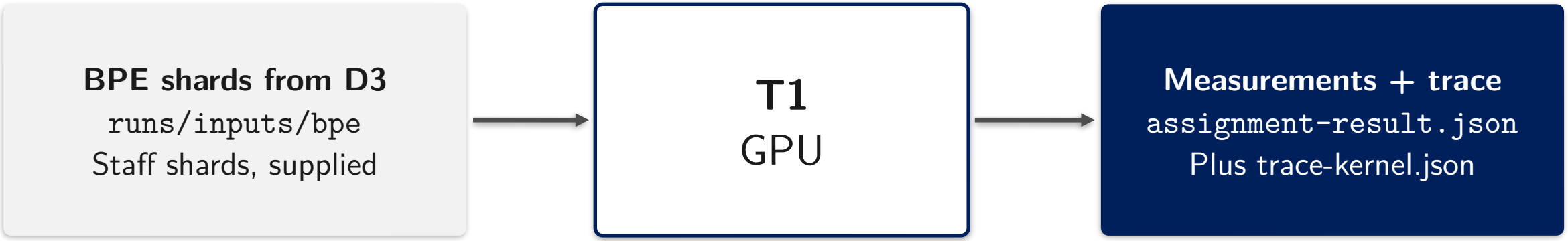
A launch returns before the work ends. Synchronize, and separate compile time from steady state.

3

Frameworks hide costs

Eager mode, torch.compile and torch.func.vmap run the same math with different launch and compile costs.

What goes in, what comes out



Files you produce

`kernel_sweep`
Eager, compiled and your kernel at four sizes, with GB/s

`training`
Setup, warmup, first and steady step time per mode

`trace-kernel.json`
PyTorch profiler trace; open in Perfetto

Predict with the roofline model

1

Bytes

Read x and b, write y: $4 \cdot (2 \cdot R \cdot C + C)$.

→

2

Operations

One add + one tanh per element: $2 \cdot R \cdot C$.

→

3

Intensity

≈ 0.25 operations per byte.

→

4

Roofline

Far below the ridge point ($\approx$ 12 op/B on an H200).

→

5

Verdict

Memory-bound: fusing saves a round trip.

Key idea

Eager PyTorch writes x + b to memory and reads it back for tanh. A fused kernel keeps it on the chip.

One program per row block, with a mask

```
row  = tl.program_id(0)
cols = tl.program_id(1) * BLOCK
       + tl.arange(0, BLOCK)
mask = cols < n_columns
x = tl.load(X + row*stride + cols,
            mask=mask, other=0.0)
z = x + b
y = 1 - 2 / (tl.exp(2*z) + 1)
tl.store(Y + ..., y, mask=mask)
```

1 **Grid**

(rows, cdiv(columns, BLOCK)) programs run at once.

2 **Mask the last block**

Lanes past the end neither load nor store.

3 **Respect strides**

Make a transposed x contiguous in the wrapper.

4 **Stable tanh**

exp overflow gives exactly ± 1, never NaN.

Three ways to run the same model

Eager

`torch.tanh(pre + b1)`

Two kernels; easy to read.

Kernel

`BiasTanh.apply(pre, b1)`

Your fused kernel; backward supplied.

Compiled

`torch.compile(forward)`

Traced and fused for you; pays compile time once.

Key idea

Report warmup (compile) and steady-state step time separately. torch.func.vmap batches a one-example function; torch.compile compiles it.

What you write, what you get

You write

`bias_tanh`, `bias_tanh_kernel`, `measure`

`starter/kernels.py`

`forward`

`starter/workload.py`

Provided

- Model, autograd, AdamW, data loader
- Kernel backward pass (BiasTanh)
- Training and profiler harness
- Guided vmap script

Ground rules

Keep the signatures · don't edit `starter/pipeline_lab` · run through `course.py` or `lab.py`.

Build it in this order

1	2	3	4	5
Predict	**Kernel + wrapper**	**Timer**	**Forward**	**Measure**
Bytes, operations, intensity.	Grid, mask, strides, stable tanh.	Synchronize; first call separate.	Use the kernel inside the model.	Eager vs kernel vs compiled; trace; vmap.
	`check t1 --checkpoint --systems`	`same (checkpoint)`	`check t1 --systems`	`lab.py t1`

Check

Done when: All 12 tests pass on a GPU (9 checkpoint tests); one bottleneck is explained with a trace.

If something goes wrong

Wrong values in the last columns
Check: The mask, and other=0.0 in tl.load.

Wrong values for a transposed input
Check: Strides: make x contiguous when x.stride(1) != 1.

NaN for large inputs
Check: Use $1 - 2/(\exp(2z)+1)$.

Times in microseconds for big work
Check: Synchronize before stopping the clock.

Kernel slower on tiny inputs
Check: Expected: launch overhead dominates.

First compiled step takes seconds
Check: That is compilation; report it separately.

The performance study

When does fusion pay off, and what does compilation cost?

Predict first

Fused traffic is $4 \cdot (2RC + C)$ bytes; eager moves about twice as much. Achieved GB/s should approach the HBM bandwidth only for large inputs.

Input size
four sizes in `kernel_sweep`

Batch 64 / 256 / 1024
fixed model

Hidden 64 / 256
fixed batch

How to measure: One warmup + three repeats. Separate compile, launch and steady state; name one bottleneck from the trace.

How you are graded

20	40	16	24
Checkpoint	Final correctness	Performance	Analysis

Correctness: kernel and wrapper 14 · timing harness 8 · forward and gradients 10 · training in three modes 8

Check

Done when: All 12 tests pass on a GPU (9 checkpoint tests); one bottleneck is explained with a trace.

Your first ten minutes

```
$ bash setup.sh
$ source .venv/bin/activate
$ python course.py preflight
$ python course.py check t1 --checkpoint \
    --systems
$ python lab.py t1 --input runs/inputs/bpe \
    --output runs/t1-first --steps 40
$ python 04_t1_numpy/vmap_guided.py \
    --input runs/inputs/bpe
```

Your folder

`04_t1_numpy/`

Handout:
`04_t1_numpy/README.md`